

Project Title

Full Stack Notes Management Application

1. Project Overview

This project is a **Full Stack Notes Management Application** developed using **React.js, Node.js, Express.js, and MongoDB**. The application allows users to securely create, view, edit, and delete personal notes after authentication. It demonstrates practical implementation of MERN stack concepts such as authentication, CRUD operations, RESTful APIs, and frontend–backend integration.

2. Objectives of the Project

- To develop a secure notes management system
 - To implement user authentication and authorization
 - To perform CRUD operations on notes
 - To understand frontend and backend integration
 - To follow professional project structure and best coding practices
-

3. Technology Stack

Frontend

- React.js (with Hooks)
- React Router DOM
- CSS

Backend

- Node.js
- Express.js

Database

- MongoDB
- Mongoose

4. Project Architecture

The application follows a **Client–Server Architecture**:

- Frontend (React) handles user interface and interactions
- Backend (Node + Express) manages APIs and authentication
- MongoDB stores user and notes data securely

5. Core Features

- User authentication (Login & Register)
 - Create new notes
 - Edit existing notes
 - Delete notes
 - View all saved notes
-

6. Functional Modules

6.1 User Authentication Module

- User registration
- User login
- Protected routes using authentication middleware

6.2 Notes Management Module

- Add new notes
 - Edit note content
 - Delete notes
 - Display all notes in dashboard
-

7. Project Directory Structure

Backend Structure

```
notes-app/  
├── backend/  
│   ├── server.js  
│   ├── config/  
│   │   └── db.js  
│   ├── models/  
│   │   ├── User.js  
│   │   └── Note.js  
│   ├── routes/  
│   │   ├── authRoutes.js  
│   │   └── noteRoutes.js  
│   ├── controllers/  
│   │   ├── authController.js  
│   │   └── noteController.js  
│   └── middleware/  
│       └── authMiddleware.js
```

Frontend Structure

```
frontend/
├── src/
│   ├── components/
│   │   ├── NoteCard.jsx
│   │   ├── NoteForm.jsx
│   │   └── Navbar.jsx
│   ├── pages/
│   │   ├── Login.jsx
│   │   ├── Register.jsx
│   │   └── Dashboard.jsx
│   ├── App.jsx
│   └── main.jsx
└── package.json
```

8. User Interface Description

Dashboard Page

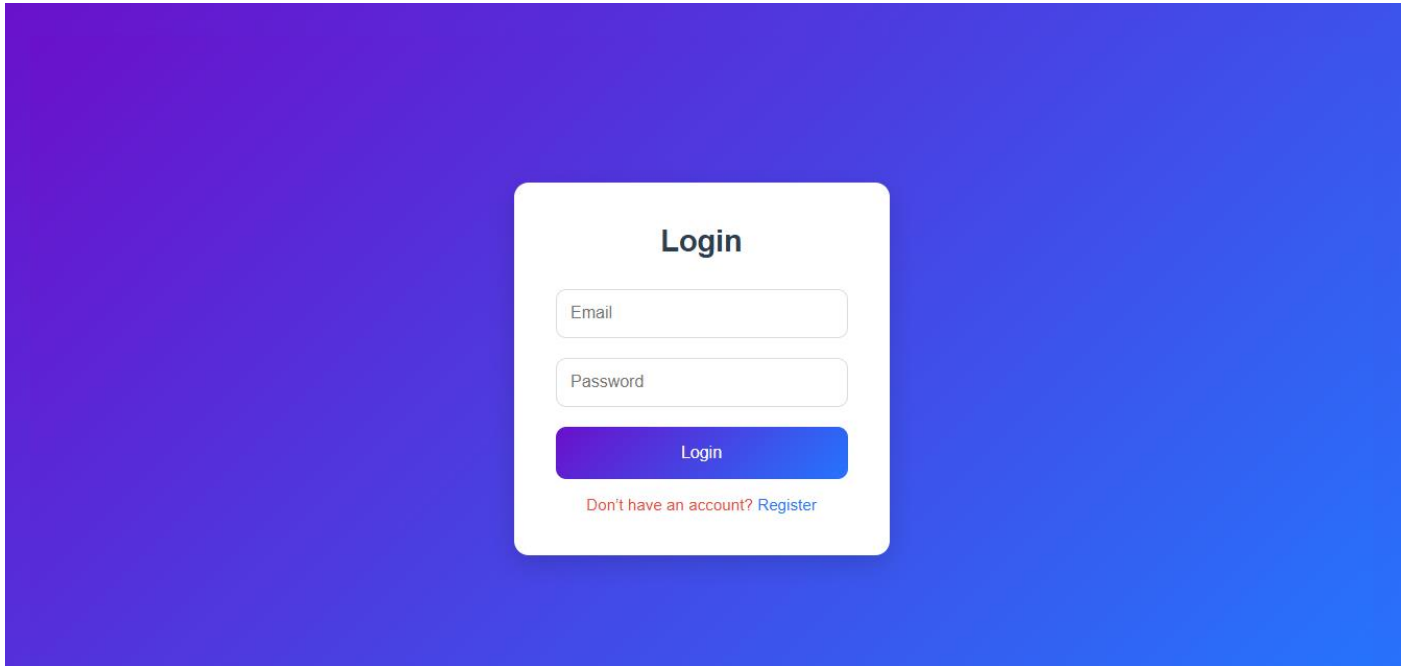
- Display all notes
- Options to add, edit, and delete notes

The screenshot displays the COSMOS Dashboard Page. At the top right, there is a 'Logout' button. The main section is titled 'My Notes' and contains a form with 'Title' and 'Content' input fields, followed by an 'Add Note' button. Below the form are four cards:

- Shopping List:** Contains a list of items (Milk, Bread, Vegetables, Fruits, Cooking Oil, Rice, Toothpaste) and 'Pin' and 'Delete' buttons.
- Important Phone Numbers:** Contains contact information (Electrician: 9876543210, Plumber: 9123456780, Internet Support: 1800-123-456) and 'Unpin' and 'Delete' buttons.
- Healthy Morning Routine:** Contains a paragraph about a healthy morning routine and 'Unpin' and 'Delete' buttons.
- Daily Task Reminder:** Contains a paragraph about completing pending work and 'Pin' and 'Delete' buttons.

Login page

- User login form with validation

A login form UI mockup centered on a blue gradient background. The form is a white rounded rectangle with the title "Login" at the top. It contains two input fields: "Email" and "Password". Below these is a blue "Login" button. At the bottom, there is a link that says "Don't have an account? Register".

Login

Email

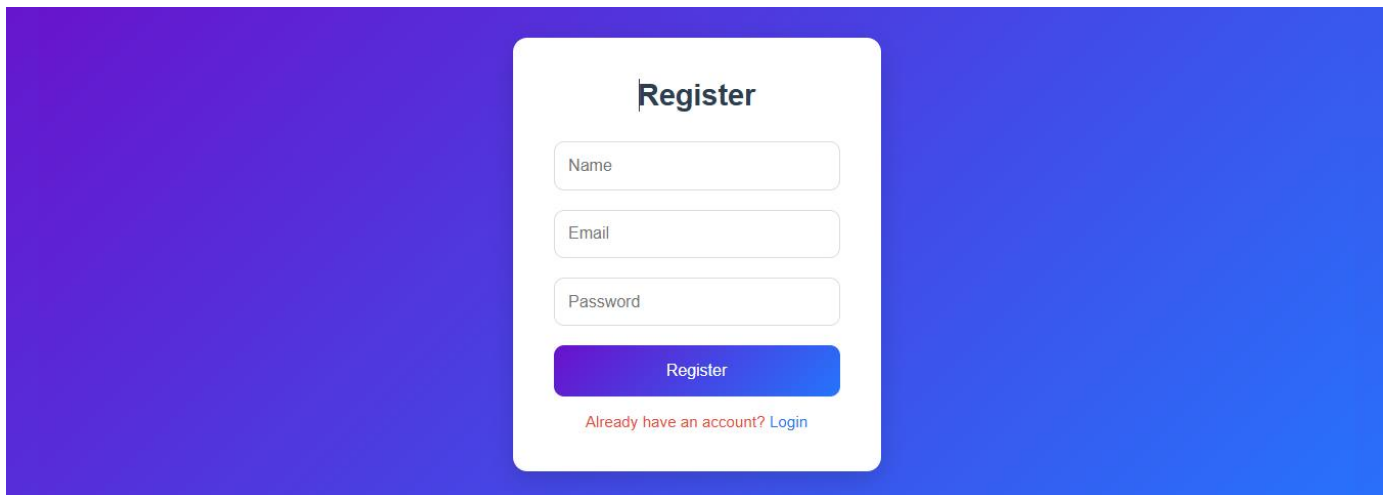
Password

Login

Don't have an account? [Register](#)

Register Page

- New user registration form

A registration form UI mockup centered on a blue gradient background. The form is a white rounded rectangle with the title "Register" at the top. It contains three input fields: "Name", "Email", and "Password". Below these is a blue "Register" button. At the bottom, there is a link that says "Already have an account? Login".

Register

Name

Email

Password

Register

Already have an account? [Login](#)

9. Security & Validation

- Authentication middleware for protected routes
- Input validation on frontend and backend
- Secure database connection using environment variables